

Technical Summative Assessment 1

Modules 1 and 2 | Console Library-Request Prototype

SCOPE	SCORE	PRODUCT	SUGGESTED TIME
Modules 1-2	100 points	Console prototype	2.5 hours

PROJECT CONTINUITY: Save the project as BookBridge_TSA1. Preserve the field names, membership rules, decision logic, sample transactions, and test evidence because Technical Summative Assessment 2 must redesign this same behavior using objects.

Project Scenario

BookBridge is a proposed library loan and reservation management system. Build and debug a console prototype that evaluates borrowing requests, applies membership rules, estimates late-return charges, stores five transactions, and presents an operational summary. The verified rules and test data become the baseline for the next milestone.

Learning Targets

- Analyze how the JDK, JRE, JVM, source code, bytecode, and IDE support the Java build-run-debug cycle.
- Select suitable data types, constants, operators, String operations, arrays, and formatted input/output.
- Design decisions, loops, and reusable methods that implement library borrowing rules.
- Diagnose compilation, runtime, and logical errors with compiler and debugger evidence.

General Directions

- Work from the previously approved BookBridge milestone and keep a backup before refactoring.
- Analyze requirements before coding. Plan responsibilities, data flow, edge cases, and evidence.
- Use the exact project name unless the instructor supplies a section naming convention.
- Credit depends on demonstrated behavior, source quality, evidence, and explanation - not screenshots alone.
- Disclose AI assistance, verify suggestions, and retain an AI-use log when required.

Technical Requirements and Point Allocation

1. Project Setup and Platform Evidence - 15 points

- Create a Java project named BookBridge_TSA1 with a main class named BookBridgeApp.
- In a short README, trace .java source -> compiler -> .class bytecode -> JVM -> output and identify what the JDK contributes.
- Provide a successful clean-build/run record from NetBeans or JDeveloper and identify one IDE debugging feature used.

2. Transaction Input and Validation - 20 points

- Capture member ID, member name, membership type, ISBN, book title, requested loan days, availability, and late days.
- Use STUDENT, FACULTY, and GUEST as the permitted membership types. Trim required text and preserve ISBN/member IDs as String values.
- Reject blank fields, negative late days, and requested days outside 1-30. Re-prompt after ordinary invalid input.

3. Borrowing Rules and Control Flow - 30 points

- Use switch or enhanced switch for policy: STUDENT maximum 14 days and PHP 5 late fee/day; FACULTY 30 days and PHP 2/day; GUEST 7 days and PHP 10/day.
- Classify a request as APPROVED when the book is available and requested days are within the limit; RESERVED when unavailable; otherwise DENIED.
- Accept exactly five valid requests using a loop and coordinated arrays. Display a formatted transaction result after each request.
- Report outcome counts, the request with the greatest requested days, and the total estimated late charge.

4. Method Decomposition and Code Quality - 20 points

- Implement and call at least five focused methods, including validateMembershipType, getMaximumDays, getDailyFine, classifyRequest, and printSummary.
- Use parameters and return values to separate calculation, validation, and presentation. Avoid unnecessary global state.
- Use meaningful names, consistent indentation, named constants, and comments that explain non-obvious decisions.

5. Testing and Debugging Evidence - 15 points

- Document at least five tests: normal approval, unavailable reservation, over-limit denial, boundary value, and summary calculation.
- Identify one compilation, runtime, or logical defect and show how compiler output, a breakpoint, stepping, or variable inspection led to the fix.
- Record input, expected result, actual result, and pass/fail status for every test.

Non-Negotiable Constraints

- Use only Module 1 and 2 concepts; do not replace the coordinated arrays with objects, collections, or a database yet.
- Do not accept a transaction until all required values are valid, and do not terminate after an ordinary input mistake.
- The submitted program must be explainable line by line during an instructor spot check.

Submission Package

Required files

- Complete BookBridge_TSA1 project folder or ZIP.
- README.pdf or README.docx with platform and build/run evidence.
- TSA1_Test_Report.pdf or DOCX containing five tests and the debugging record.

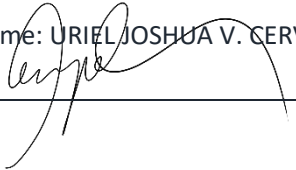
Required evidence

- Source compiles and runs without errors.
- Console transcript contains five accepted transaction inputs and the final summary.
- Expected and actual results are shown for all tests.
- One defect is classified, diagnosed, and corrected with evidence.

Student Declaration

I certify that I can explain the submitted code, identify any AI-assisted portions, and reproduce the demonstrated behavior during the instructor check.

Student name: URIEL JOSHUA V. CERVANIA _____ Section: TA21 _____

Signature:  _____ Date: 9-28-2026 _____

Analytic Rubric - 100 Points

Use the highest descriptor fully supported by the submitted evidence. Award points within the criterion maximum; descriptors are calibration anchors.

Criterion	Exemplary	Proficient	Developing	Beginning	Max
Platform setup and evidence	Accurate execution trace, clean project, reproducible build/run, and justified debugger use.	Project runs and explanation is mostly accurate; minor evidence gap.	Project is inconsistent or platform explanation contains several gaps.	No reliable build/run or major misunderstanding of JDK/JRE/JVM.	15
Input model and validation	Data types/constants fit the domain; all required validation recovers cleanly.	Most inputs and validation work; one minor edge case missed.	Several invalid values are accepted or recovery is inconsistent.	Inputs are largely unvalidated or common mistakes terminate execution.	20
Rules and control flow	All policies, outcomes, five-request loop, arrays, and summaries are correct.	Core behavior works with one minor rule/report defect.	Partial implementation with several incorrect decisions or summaries.	Core rules or repeated processing are missing or mostly incorrect.	30
Methods and code quality	Focused methods, clear parameters/returns, good names, constants, and useful comments.	Adequate decomposition with limited duplication or naming issues.	Large methods, duplication, or weak separation reduces clarity.	Monolithic and difficult-to-follow code; required methods absent.	20
Testing and debugging	Five purposeful tests and convincing debugger-based diagnosis prove correctness.	Required tests and correction evidence are generally adequate.	Too few/weak tests or diagnosis is asserted without proof.	Little testing and no credible debugging evidence.	15

SCORING RULE: The five criterion maxima total 100 points. No extra credit should raise a score above 100.